Assignment 1

Design and Implementation of MLOps for Red Wine Quality Prediction

Authors

Group 3: Andrea Ciavatti, Csanád Végh, Leopold Paris, Niels Boonstra, Vladimir Popov

Course

JM0140-M-6 Data Engineering

Institution

Jheronimus Academy of Data Science

October 29, 2024

JADS Jheronimus Academy of Data Science

Contents

1 Overview of the ML Application **2**
- 1.1 Description of the ML Application . . . 2
- 1.2 Goals and Requirements . . . 3
- 1.3 MLOps Requirements . . . 3
  - 1.3.1 Retraining Requirements . . . 3
  - 1.3.2 Model Monitoring Requirements . . . 3
  - 1.3.3 Model Deployment Constraints . . . 3

2 Design and Implementation of the MLOps System **4**
- 2.1 ML Pipelines . . . 4
- 2.2 Prediction/Serving Components . . . 5
- 2.3 CI-CD Pipelines . . . 5

3 Reflection on the Design and Implementation of the MLOps System **5**
- 3.1 Challenges and Solutions . . . 6

4 Individual Contributions of Students **7**

1 Overview of the ML Application

1.1 Description of the ML Application

The Red Wine Quality Prediction application is designed to classify red wines into three quality categories: good, medium, or bad. This classification is based on the wines' physicochemical properties, utilizing a dataset from the UCI Machine Learning Repository [1]. The dataset comprises various measurements of red wine samples, including acidity levels, alcohol content, and sugar concentrations.

Each wine sample in the dataset includes several explanatory variables that serve as inputs for the machine-learning models. These variables are described in table 1:

Variable Name	**Description**
Fixed Acidity	Concentration of non-volatile acids in the wine. Contributes to the tartness and stability of the wine.
Volatile Acidity	Concentration of acetic acid, which, at high levels, gives an undesirable vinegar flavor to the wine.
Citric Acid	Adds freshness and enhances the acidity of the wine, similar to what citrus fruits do.
Residual Sugar	Amount of sugar left after fermentation. Plays a significant role in the sweetness of the wine.
Chlorides	Amount of salt in the wine, contribute to its overall taste and perception.
Free Sulfur Dioxide	A measure of the sulfur dioxide available to prevent microbial growth and oxidation.
Total Sulfur Dioxide	Total concentration of SO2, both bound and free, in the wine. Used as a preservative.
Density	Relative density of the wine, which can provide insight into sugar content and alcohol levels.
pH	Indicates the acidity of the wine, impacting its stability and taste.
Sulphates	Contributes to the antimicrobial properties of the wine, helping to preserve it.
The alcohol	Alcohol content of the wine influences its body, flavor, and aroma.

Table 1: Descriptions of wine quality prediction variables.

By employing machine learning techniques, the application aims to provide a scalable and efficient solution for rapid wine quality assessment. This tool supports decision-making processes in the wine industry, enabling producers and distributors to maintain consistent quality control based on measurable attributes.

1.2 Goals and Requirements

The primary goal of the Red Wine Quality Prediction application is to deliver accurate and timely predictions of wine quality to assist in quality control and decision-making within the wine industry. To achieve this, the application must meet the following requirements:

- **High Prediction Accuracy**: The model should achieve a high level of classification accuracy to ensure its predictions are reliable and align with real-world assessments.
- **Adaptability and Scalability**: The application must be capable of adapting to new data and changes in wine profiles, maintaining its performance over time.
- **Efficient Retraining Mechanisms**: Implement automated processes for regular retraining of the model to incorporate new data and improvements in the machine learning pipeline.
- **Timely Deployment**: Ensure that updates to the model and pipeline are deployed promptly so that users have access to the most recent and accurate predictions.

1.3 MLOps Requirements

To fulfill the application's goals and maintain its effectiveness, several Machine Learning Operations (MLOps) requirements have been established:

1.3.1 Retraining Requirements

Given the dynamic nature of wine production and the potential for shifts in data distributions, the model requires regular retraining to remain accurate. The application includes an automated retraining process scheduled to occur weekly. This retraining is triggered by a scheduled event, allowing the model to learn from any newly available data and adjust to changes in wine characteristics without manual intervention.

1.3.2 Model Monitoring Requirements

Continuous monitoring of the model's performance is crucial to detect any degradation over time. Evaluation metrics such as accuracy were calculated during the model evaluation phase. These metrics help assess whether the model meets the predefined performance thresholds necessary for deployment. Monitoring also involves checking for data drift and concept drift to ensure that the model remains valid as the underlying data changes.

1.3.3 Model Deployment Constraints

The deployment of the model must consider several constraints to ensure smooth operation in a production environment:

- **Integration with Cloud Services**: Since the application handles data stored in Google Cloud Storage, it integrates with Google Cloud's Vertex AI and Cloud Build services for model deployment. This setup provides a controlled, containerized environment suitable for production use.

- **Resource Optimization**: Deployment should be optimized for resource usage to handle varying workloads efficiently, ensuring scalability.
- **Compliance and Security**: Although the dataset does not contain personally identifiable information, the application must adhere to data security best practices and comply with any relevant regulations to protect data integrity.

2 Design and Implementation of the MLOps System

2.1 ML Pipelines

The MLOps system is configured according to the design illustrated in Figure 1, incorporating three key components: an ML pipeline, a prediction UI, and a CI/CD pipeline.

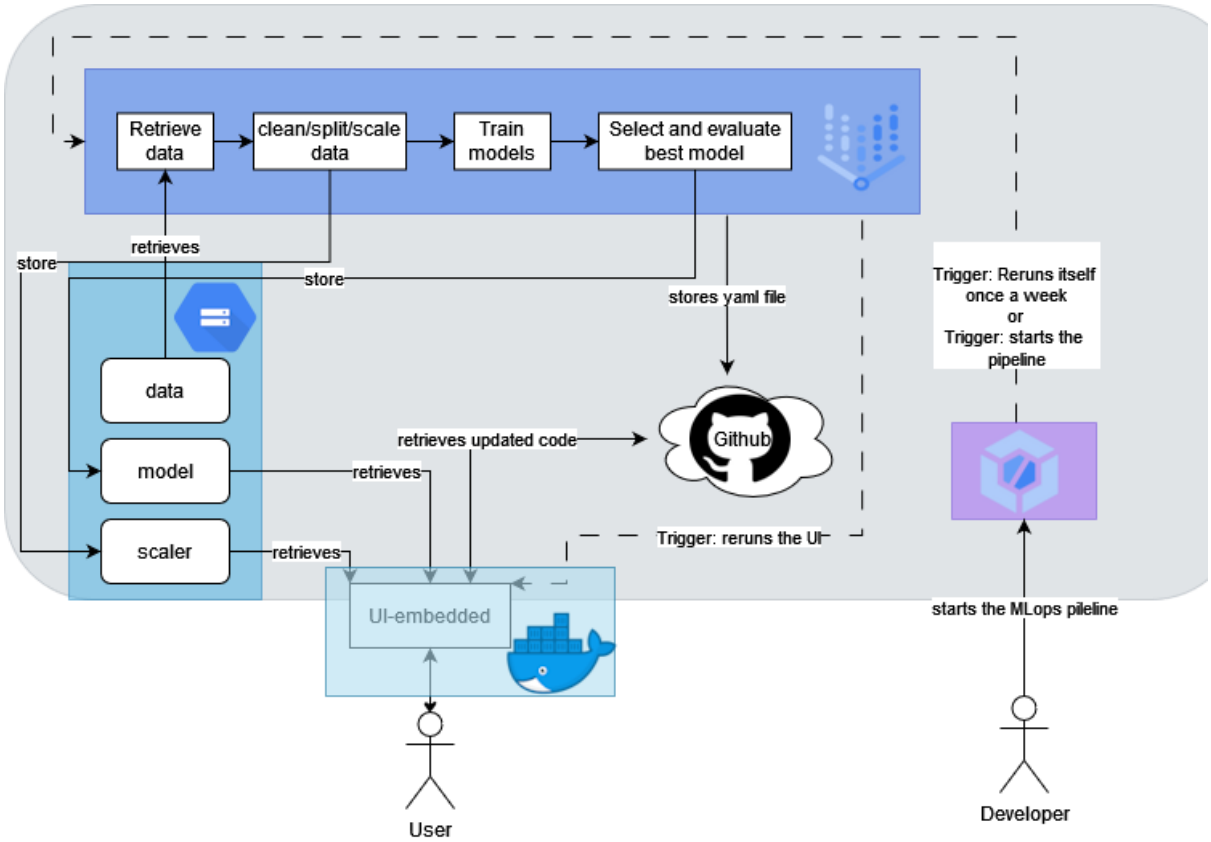


Figure 1: The ML Pipeline Design

The ML pipeline is divided into two main parts: data preparation and modeling. In the data preparation phase, data is extracted from a Google Storage Bucket. Provided the Google Cloud server isn't down this data will afterward be cleaned and split into test and training datasets. The first component extracts data from the storage bucket and converts it into a CSV file and this will be downloaded using Blob. Afterward, the data will be split into an 80/20 test split and will send this data forward to the preprocessing part. Here the data will be scaled to retrieve better results during the machine learning phase and this scaler will be sent into the bucket to be used by the prediction UI later.

The modeling phase of the ML pipeline runs across three components, each fitting a different model in the following order: KNN, decision tree, and SVM. The pipeline logs accuracy scores on the training data to ensure the best model is chosen. Afterward, the scaled and cleaned test dataset is sent to all three models and compared to the accuracy scores. The model with the highest accuracy is selected for the serving component if a certain threshold has been reached by this model, if it hasn't reached this threshold then this model won't be deployed. The model will be deployed by sending it into a Google bucket and releasing a trigger to run the Prediction-ui again.

2.2 Prediction/Serving Components

Following the ML pipeline, the prediction/serving component retrieves the selected model from the bucket. The model is encoded using Python's `pickle` library [2] and loaded by the prediction service during initialization. This component features a user interface built with Flask [3] that allows access to the model. To minimize the amount of applications an embedded design had been chosen. This makes sure that the UI that the user interacts with also contains the prediction of the model part. Normally you would use an API but the embedded version makes it easier for this to be run and debugged. The User will be able to insert its data into the UI and the embedded app will retrieve the model and scaler from the Google bucket to make an accurate prediction.

2.3 CI-CD Pipelines

The CI-CD part of this MLOps system consists of two types of triggers. The CI/CD pipeline is configured using Google Cloud Build. Event-based triggers are set up to initiate a rebuild and deployment of the prediction service whenever a new model is stored in the Google Cloud Storage bucket. This ensures that the prediction service always uses the latest model. Scheduled triggers are configured using Cloud Scheduler to re-execute the ML pipeline every week, ensuring the model stays up-to-date with any new data or changes in the codebase. This periodic retraining allows the model to adapt to any shifts in data distributions over time. By automating these processes, we reduce the need for manual intervention and minimize the risk of human error. If desired, the user can also manually rerun the pipeline in Vertex AI to update the model or modify pipeline components.

3 Reflection on the Design and Implementation of the MLOps System

The design and implementation of the MLOps system has unfortunately not been perfect and several areas of improvement have been listed in this segment. One significant area for improvement is updating the user interface (UI) to give it a more modern look and feel. Upgrading the UI can enhance user engagement, streamline interactions, and improve overall usability. Currently, the UI exists as a barebones MVP that could easily be improved.

Another enhancement involves the introduction of a Developer Mode in the UI that allows users to retrain models by rerunning the entire MLOps pipeline. This feature makes it possible for developers and data scientists to experiment with different configurations and datasets. This will make it easier to get a more hands-on approach to model development.

To ensure the robustness of new models, we propose deploying them in a shadow mode instead of just replacing the model. This approach allows the new model to run in parallel with the existing model, generating predictions without affecting user experience. By monitoring the performance of both models side by side, we can ensure that the new model meets or exceeds the accuracy of the original before fully transitioning. This risk mitigation strategy not only boosts confidence in deploying updates but also provides a safety net should any issues arise.

Lastly, separating the UI from the API could be a step toward enhancing development efficiency. By embedding the UI within a dedicated API, we create a modular architecture that allows teams to work on both components simultaneously without interference.

3.1 Challenges and Solutions

The design and implementation of the MLOps system were not without challenges, which tested our adaptability and problem-solving skills. One major hurdle was the unexpected downtime of Google Cloud services last Friday. When the server went down it delayed exactly the part that was being worked on. This experience underscored the importance of having a robust backup plan and highlighted the need for a more resilient architecture that could handle such outages. To address the downtime of Google Cloud services, we transitioned to an alternative server.

Another significant challenge arose when we decided to switch from a TensorFlow model to a Sklearn model[4]. While this change aimed to diversify the model selection process, it required us to revisit and refactor various parts of the code. Adapting the pipeline to accommodate the different requirements and functionalities of Sklearn turned out to be hard since more code needed to be changed than expected. Changing from Skelar to Pickle is one part that gave a bunch of errors. The transition from TensorFlow to Sklearn was also resolved by modifying our approach to the application architecture. We shifted from a separate UI and API structure to an embedded UI application, which allowed Sklearn models to run more smoothly within our workflow.

Finally, practical obstacles came when we were limited to working on one laptop in the project's final stages. This made it difficult for each member to work on it continuously, and we had to carefully coordinate work times. It also limited our capacity to properly use version control, potentially leading to conflicts and additional problems in handling updates. We used GitHub's collaboration tools, such as pull requests and code reviews, to allow all team members to contribute code simultaneously. Despite hardware limitations, this method allowed us to retain productivity and code quality.

4 Individual Contributions of Students

Our project was a collaborative effort where each team member took on specific roles and responsibilities to ensure the successful completion of the MLOps system.

- **Andrea Ciavatti**
  Led machine learning pipeline development in Vertex AI, including data preprocessing, feature engineering, and implementing KNN, Decision Tree, and SVM models. Set up evaluation metrics and model selection logic.

- **Csanád Végh**
  Focused on deployment. Containerized the selected model using Docker and integrated it with Google Cloud Run. Set up Google Cloud Storage for model storage and retrieval.

- **Leopold Paris**
  Developed the user interface using Flask. Designed the embedded UI for user input and predictions, ensuring integration with the backend prediction service.

- **Niels Boonstra**
  Managed the CI/CD pipelines. Configured Google Cloud Build triggers for automated rebuilding and redeployment whenever new models were available or code changes were pushed to GitHub.

- **Vladimir Popov**
  Managed project documentation and coordination. Wrote substantial chunks of the report and helped debug issues such as model transitions and cloud service disruptions.

References

[1] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision support systems*, vol. 47, no. 4, pp. 547–553, 2009.

[2] "pickle — Python object serialization." [Online]. Available: https://docs.python.org/3/library/pickle.html

[3] "Welcome to Flask — Flask Documentation (3.0.X)." [Online]. Available: https://flask.palletsprojects.com/en/stable/

[4] "Documentation scikit-learn: machine learning in Python — scikit-learn 0.21.3 documentation." [Online]. Available: https://scikit-learn.org/0.21/documentation.html